Practical – 8

Practical Title: PHP Form Processing with Server-Side Validation and CSV/JSON File Storage

Problem Definition: Process submitted registration/contact form data using PHP. Validate and sanitize inputs on the server side and store records in CSV or JSON file format. Display success/error messages.

A. Theory

1. Is the form submitted using POST?
Yes. The registration form uses the POST method to send user details to the PHP server-side script. POST is preferred because the submitted information is sent in the request body rather than being displayed in the URL.

2. Are inputs validated and sanitized server side?
Yes. Server-side validation checks the submitted data, such as name, student ID, email, phone number, program, semester, and password. Input sanitization and appropriate output escaping help prevent invalid data and security issues.

3. Is file writing handled safely?
Yes. Registration details are stored in a CSV file using PHP file-handling functions. File locking with flock() helps prevent conflicts when multiple registrations are submitted simultaneously. Passwords are stored using secure password hashing rather than as plain text.

4. Are success and error responses displayed clearly?
Yes. The PHP script displays a success message when registration is completed successfully. If validation fails or a duplicate registration is detected, an appropriate error message is displayed so the user knows what needs to be corrected.

B. Implementation/ Output/Result:

In this practical, I implemented server-side form processing for the existing StudentHub Portal registration form using PHP and XAMPP. First, I configured XAMPP, started the Apache server, and placed the project folder inside the htdocs directory.

I connected the registration form to a PHP script using the POST method to send student details to the server. The PHP script performs server-side validation of inputs such as name, student ID, email, phone number, program, semester, and password. Invalid or incomplete information is handled through appropriate error messages.

After successful validation, the registration details are stored in a CSV file using PHP file-handling functions. File locking helps prevent conflicts during writing, duplicate registrations are checked, and passwords are stored securely using password hashing.

Finally, I tested the registration form through localhost. The system successfully processed the submitted details, saved the registration data, and displayed an appropriate success message.

Through this practical, I learned PHP form handling, server-side validation, file management, and integration of frontend forms with a backend server.

C. Evidence of Work (Screenshots):

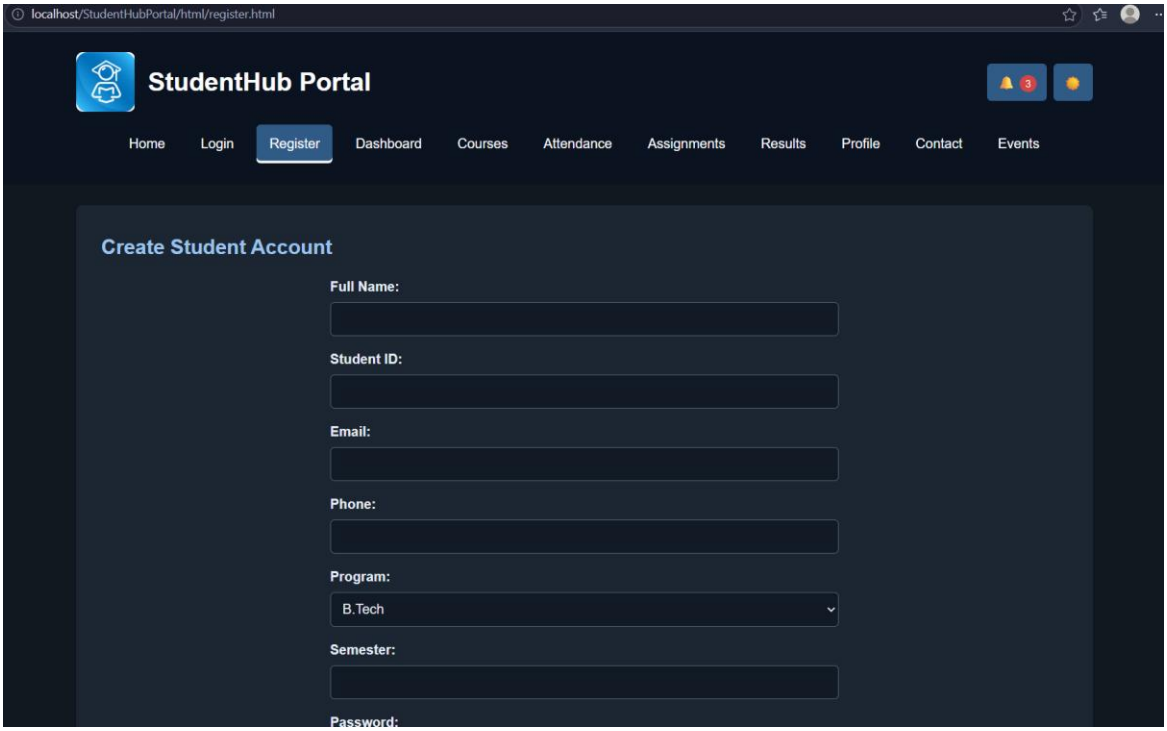


Fig1: Registration Form

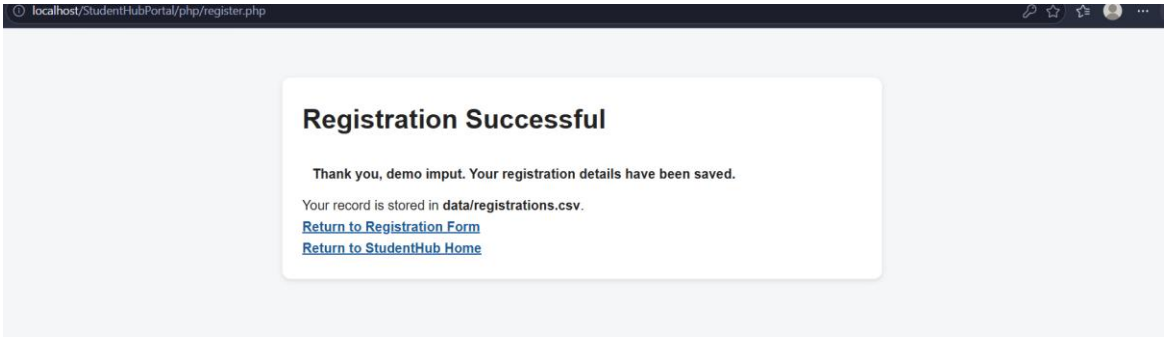


Fig2: Successful Registration

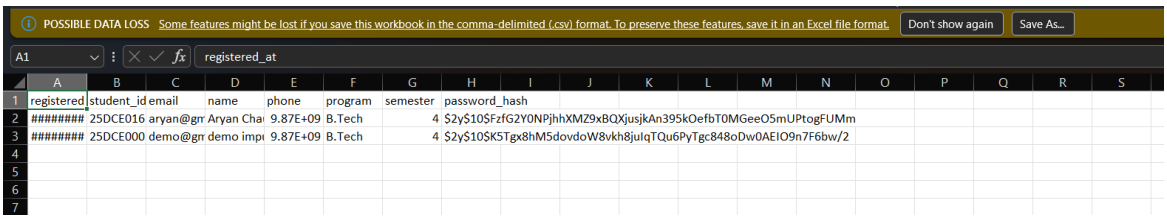


Fig3: CSV File Storage

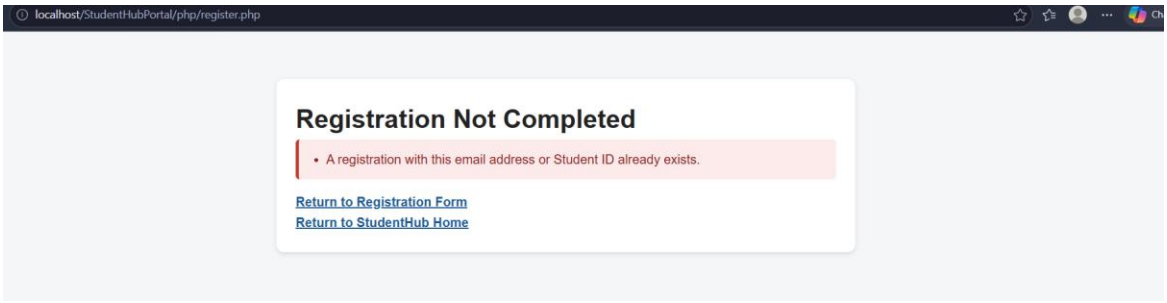


Fig4: Duplicate Registration Error

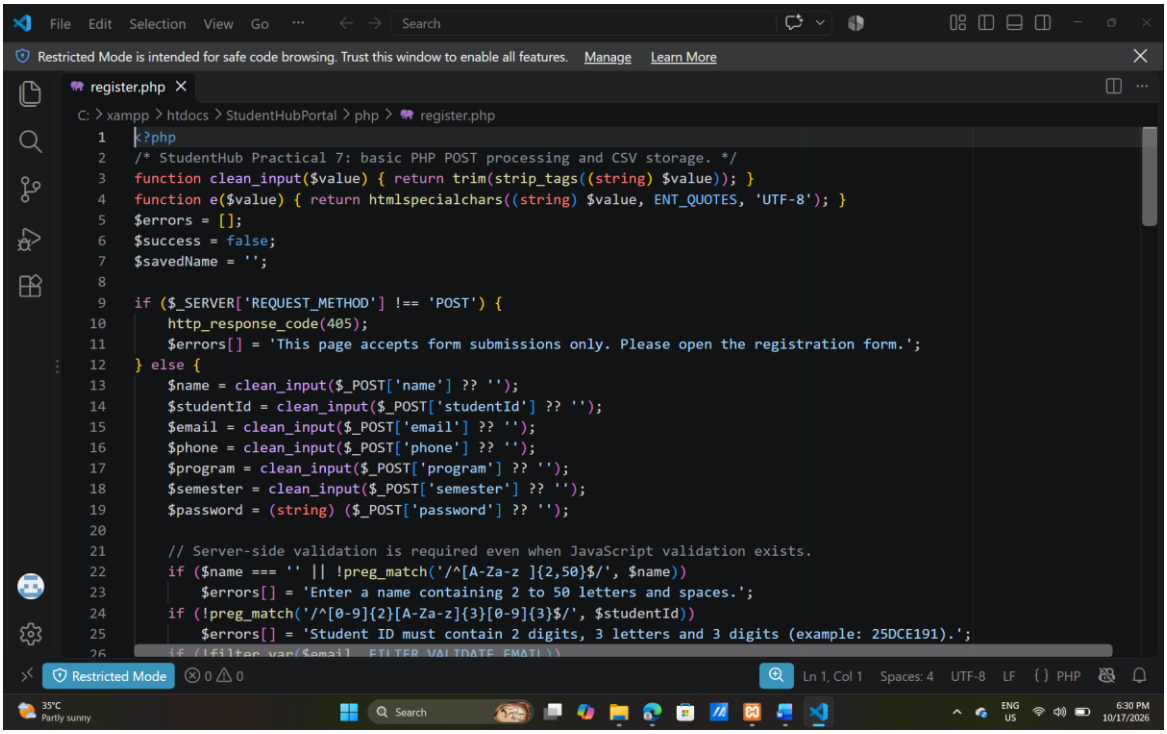


Fig5: PHP Source Code

D. GitHub Repository Link:

aryan150806-wq/StudentHubPortal: Web Development Framework P

E. Challenges Faced:

1. XAMPP Configuration: Setting up XAMPP and configuring the project inside the htdocs folder to run through localhost.

2. PHP Integration: Connecting the existing HTML registration form with the PHP backend using the POST method.

3. Server-Side Validation: Implementing validation in PHP to ensure that submitted student details are correct and complete.

4. CSV File Handling: Storing registration details in a CSV file and handling file permissions and writing operations properly.

5. Error Handling: Managing duplicate registrations and displaying appropriate success and error messages to users.

F. Learning Outcomes:

Through this practical, I successfully implemented server-side form processing using PHP and XAMPP. I learned how to connect an HTML registration form with a PHP backend using the POST method. I implemented server-side validation and sanitization to handle user inputs properly. I learned to store registration details in a CSV file using PHP file-handling functions. I also understood the importance of safe file writing and duplicate registration checking. The system successfully processed registration details and displayed appropriate success and error messages.

Faculty Remarks: